



南開大學
Nankai University

数据安全

频率隐藏 OPE 方案实现

姓名：李子恒

学号：2312114

专业：密码科学与技术

2026 年 6 月 5 日

目 录

1	实验内容	2
2	实验原理	2
2.1	OPE 与 FH-OPE	2
2.2	重复插值测试	2
2.3	B+ 树编码机制	2
3	实验环境	3
4	实验步骤	3
4.1	FH-OPE 的复现	3
4.2	重复插值测试	4
5	运行结果	5
5.1	阶段 1: 混合插入	5
5.2	阶段 2: 重复插入 'x' 及范围查询	5
5.3	阶段 3 & 4: 编码分布观察与最终结果	5
5.4	运行中的问题分析与修复	7
6	心得体会	7

1 实验内容

本次实验参照教材 7.3.3 节“FH-OPE 实现”的内容，完成频率隐藏保序加密 (Frequency-Hiding Order-Preserving Encryption, FH-OPE) 方案的复现。具体来说，需要用 C++ 实现一棵用于管理编码映射的 B+ 树，将其编译为 MySQL 的用户自定义函数 (UDF)，并在 Python 客户端中实现随机化加密和位置计算。在此基础上，通过反复插入相同明文数值的测试，观察编码树的分裂、重编码 (Recode) 过程以及频率隐藏的实际效果。

2 实验原理

2.1 OPE 与 FH-OPE

保序加密 (Order-Preserving Encryption, OPE) 是一类特殊的加密方案，它能保证密文的数值顺序与明文的顺序保持一致——对于任意两个明文 $m_1 < m_2$ ，加密后的密文满足 $c_1 < c_2$ 。这个性质让数据库可以在不解密的情况下直接对密文进行范围查询和排序，在云数据库场景下有很好的应用前景。

但传统 OPE 方案有一个明显的安全缺陷：频率泄露。相同的明文每次加密得到的密文是相同的（或高度近似的），攻击者通过观察密文的频率分布就能推断明文信息。比如一张表里“apple”出现了 100 次而“pear”只出现了 1 次，即使看不到明文，也能从密文分布猜出哪个对应“apple”。

FH-OPE 正是为了解决频率泄露问题而提出的。它的核心思路是在客户端侧和服务端侧分别做两件事：客户端使用随机化加密（本实验中是 AES-CBC 配合随机 IV），使得同一个明文每次加密产生完全不同的密文，从根源上隐藏频率信息；服务器侧维护一棵 B+ 树，为每条记录的密文分配一个叫“编码” (encoding) 的整数值，这些编码整体保持字典序，但编码之间的间隔不反映频率。简单打个比方——随机化加密好比给每个学生发一件图案完全随机的外套，谁也看不出谁和谁穿的是“同款”；而 B+ 树编码好比给每人分配一个学号，学号有序但跟外套没有半毛钱关系。

从安全性的角度，教材中证明了 FH-OPE 可以达到 IND-FAOCPA（频率分析下的有序选择明文攻击不可区分性）安全。通俗地说，就是攻击者即使可以自适应地选择明文并拿到密文和编码，也无法从密文分布中获取比“顺序”更多的信息。

2.2 重复插值测试

重复插值测试是指向系统中反复插入同一个明文（每次加密产生不同密文），观察 B+ 树编码的动态变化。这个测试在实验中非常关键，主要有三个目的：

1. **验证频率隐藏**：如果同一个明文的多个密文获得了分散在不同位置的编码值，就说明攻击者确实无法从编码分布推断频率。
2. **观察树分裂**：随着插入量增加，B+ 树的叶子节点会逐渐填满，达到阈值后触发分裂 (rebalance)，这是树结构自我维护的基本机制。
3. **观察 Recode 过程**：当某个叶子节点的编码区间 $[lower, upper]$ 不够分配新编码时（即区间宽度小于 2，没有整数可用），系统需要把该节点及相邻兄弟节点收集起来，在更宽的范围内重新均匀分配编码。这是整个 FH-OPE 方案中最核心的动态调整操作。

2.3 B+ 树编码机制

FH-OPE 的编码管理依赖于棵定制化的 B+ 树，它和数据库课程里学的经典 B+ 树大体相似但有一些针对性的设计：

节点类型：分为 InternalNode（内部节点）和 LeafNode（叶子节点）。内部节点不存数据，只记录每个子节点包含的密文数量（child_num），用于在插入和搜索时做路由导航；叶子节点存实际的密文字符串和对应的编码值，还有两个关键属性 lower 和 upper，界定该节点的编码值可用范围。

分裂 (rebalance)：当节点的数据量达到上限 $M = 128$ 时，将后半部分数据拆分到新节点，并调整父节点的索引。如果连根节点都需要分裂，就创建一个新的内部节点作为根，树高加一。

Recode (重编码)：这是 FH-OPE 特有的操作。当在某个叶子节点中分配新编码时发现区间已满 ($right - left < 2$)，系统把该节点加入“待重编码列表”，检查当前区间是否足以容纳所有密文。如果不够，就向左右兄弟节点扩展，甚至倍增上界 ($upper * 2$)，直到找到足够空间，然后在整个区间内等间隔地重新分配所有编码值。Recode 完成后会更新一个全局的 update 映射表，客户端通过存储过程批量更新数据库中受影响的记录。

叶子节点之间用 left_bro 和 right_bro 指针串成双向链表，正是为了方便 Recode 时向两侧扩展查找可用空间。

3 实验环境

实验在一台 Ubuntu 22.04 虚拟机上进行，具体软件环境如下：

- 操作系统：Ubuntu 22.04 LTS (VMware Workstation 虚拟机)
- 数据库：MySQL 8.0.36
- C++ 编译器：g++ 11.4.0
- MySQL 开发库：libmysqlclient-dev (编译 UDF 所需)
- Python 版本：3.10.12
- Python 依赖：pymysql (数据库连接)、pycryptodome (AES 加解密)

编译 UDF 动态库的命令大致如下：

```
1 g++ -fPIC -shared -o libope.so Node.cpp UDF.cpp \  
2 $(mysql_config --cflags) $(mysql_config --libs)
```

编译后将生成的 libope.so 复制到 MySQL 插件目录（可通过 `SHOW VARIABLES LIKE 'plugin_dir'` 查看），然后在 MySQL 中执行 import.sql 脚本即可完成建表和 UDF 注册。

4 实验步骤

4.1 FH-OPE 的复现

复现过程按照“数据结构 → UDF 接口 → 数据库配置 → 客户端”的顺序推进：

1. **编写 B+ 树数据结构** (Node.h 和 Node.cpp)。实现 InternalNode 和 LeafNode 两种节点类型，以及 insert、search、rebalance、Encode 和 Recode 等核心方法。其中 Encode 方法负责在叶子节点的 $[lower, upper]$ 区间内为新密文分配合适的编码值，当区间不足时触发 Recode 流程。
2. **编写 MySQL UDF 接口** (UDF.cpp)。将 B+ 树的操作封装为 5 个 MySQL 函数，供 SQL 语句直接调用：

- FHInsert(pos, ciphertext)：向 B+ 树指定位置插入密文，返回分配的编码值

- `FHSearch(pos)`: 根据位置查询编码值（用于范围查询的边界转换）
 - `FHUpdate(ciphertext)`: Recode 发生后，根据密文查询更新后的编码
 - `FHStart()` / `FHEnd()`: 返回最近一次 Recode 的编码更新范围边界
3. **数据库配置**。执行 `import.sql` 脚本，完成三件事：创建 `example` 表（含 `encoding` 和 `ciphertext` 两个字段）、注册上述 5 个 UDF 函数、创建 `pro_insert` 存储过程。存储过程的逻辑是：先调用 `FHInsert` 获取编码，将记录插入表中；如果返回值为 0（说明触发了 Recode），则用 `UPDATE` 批量修正受影响记录的编码值。
 4. **编写 Python 客户端**（`client.py`）。实现随机化 AES 加密（每次使用随机 IV 确保同一明文产生不同密文）、基于 `local_table` 频率统计的位置计算、以及 `Insert` 和 `Search` 两个核心操作的封装。
 5. **运行基础测试**。混合插入多种水果名称（`apple`、`pear`、`banana`、`orange`、`cherry`，部分重复），验证基本的插入、编码分配和范围查询是否正常工作。

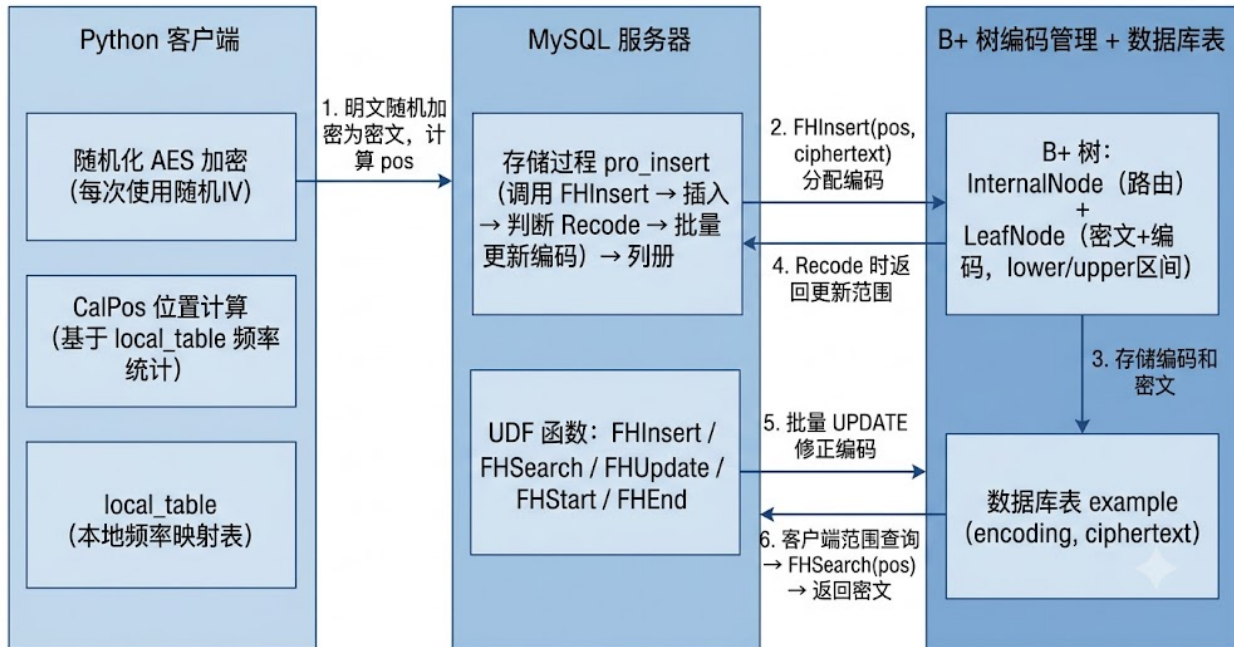


图 1: 系统整体架构

4.2 重复插值测试

在基础功能验证通过后，在 `client.py` 的 `__main__` 中分四个阶段进行测试：

1. **阶段 1——混合插入**：依次插入 `apple`、`pear`、`banana`、`orange`、`cherry`、`apple`、`cherry`、`orange` 共 8 条记录，观察 `local_table` 的频率统计和插入返回状态。
2. **阶段 2——重复插入 'x'**：循环 30 次插入 'x'，每次打印是否触发了分裂/Recode。插入完成后执行范围查询 `Search('b', 'p')`，验证在大量重复插入后范围查询仍能正确工作。
3. **阶段 3——编码分布观察**：直接查询数据库中所有记录的 `encoding` 和 `ciphertext`，在客户端解密后按编码排序打印，观察频率隐藏的实际效果。
4. **阶段 4——继续插入 'y'**：再插入 20 次 'y' 触发更多分裂，最后再次查看编码分布的变化。

5 运行结果

5.1 阶段 1：混合插入

8 条记录全部正常插入，没有触发任何分裂或 Recode。local_table 正确记录了各明文的出现频率：apple 出现 2 次、pear 出现 1 次、banana 出现 1 次、orange 出现 2 次、cherry 出现 2 次。这个阶段数据量远小于节点容量 ($M=128$)，符合预期。



```
liziheng@liziheng-virtual-machine: ~/Desktop/lab4
liziheng@liziheng-virtual-machine:~/Desktop/lab4$ python3 client1.py
已清理旧数据，开始测试。

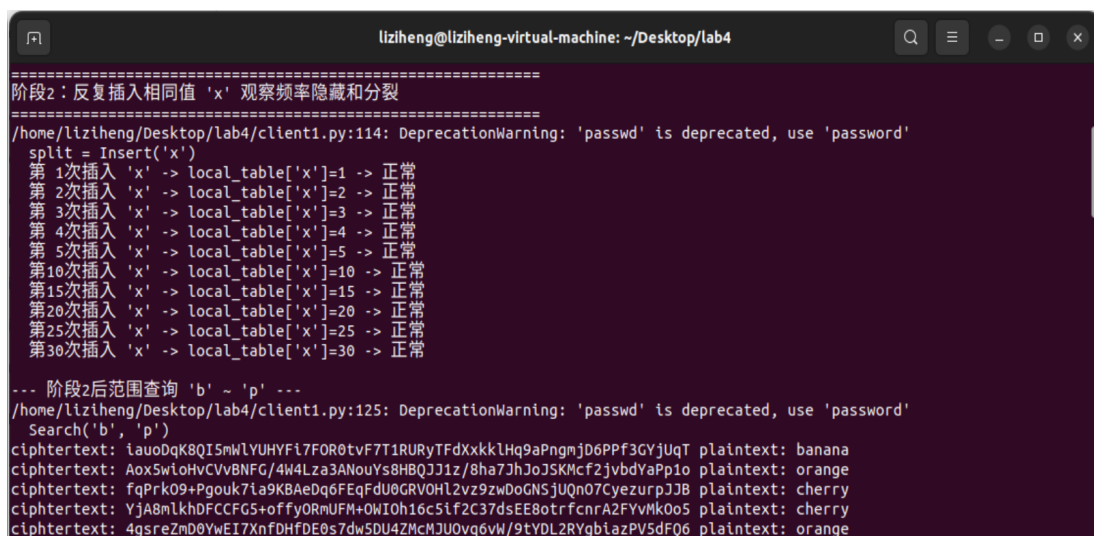
=====
阶段1：混合插入多类型数据
=====
/home/liziheng/Desktop/lab4/client1.py:103: DeprecationWarning: 'passwd' is deprecated, use 'password'
split = Insert(ciphertext)
插入 'apple' -> local_table频次: {'apple': 1} -> 正常
插入 'pear' -> local_table频次: {'apple': 1, 'pear': 1} -> 正常
插入 'banana' -> local_table频次: {'apple': 1, 'pear': 1, 'banana': 1} -> 正常
插入 'orange' -> local_table频次: {'apple': 1, 'pear': 1, 'banana': 1, 'orange': 1} -> 正常
插入 'cherry' -> local_table频次: {'apple': 1, 'pear': 1, 'banana': 1, 'orange': 1, 'cherry': 1} -> 正常
插入 'apple' -> local_table频次: {'apple': 2, 'pear': 1, 'banana': 1, 'orange': 1, 'cherry': 1} -> 正常
插入 'cherry' -> local_table频次: {'apple': 2, 'pear': 1, 'banana': 1, 'orange': 1, 'cherry': 2} -> 正常
插入 'orange' -> local_table频次: {'apple': 2, 'pear': 1, 'banana': 1, 'orange': 2, 'cherry': 2} -> 正常
```

图 2: 阶段 1：混合插入输出

5.2 阶段 2：重复插入‘x’及范围查询

30 次插入‘x’以及后续 20 次插入‘y’的过程中始终没有发生分裂，总共 0 次分裂。原因在于初始编码空间非常大 ($[0, 2^{62}]$)，50 多个编码完全不会耗尽区间；每次取区间中点，间隔非常充裕；同时总记录数远低于节点容量 128。这也从侧面说明，在正常使用场景下 Recode 并不会频繁触发。

阶段 2 结束后执行 Search(‘b’, ‘p’)，成功返回了 banana、cherry (2 条)、orange (2 条) 共 5 条记录的密文，且解密正确，说明重复插入没有破坏范围查询功能。



```
liziheng@liziheng-virtual-machine: ~/Desktop/lab4

=====
阶段2：反复插入相同值 'x' 观察频率隐藏和分裂
=====
/home/liziheng/Desktop/lab4/client1.py:114: DeprecationWarning: 'passwd' is deprecated, use 'password'
split = Insert('x')
第 1次插入 'x' -> local_table['x']=1 -> 正常
第 2次插入 'x' -> local_table['x']=2 -> 正常
第 3次插入 'x' -> local_table['x']=3 -> 正常
第 4次插入 'x' -> local_table['x']=4 -> 正常
第 5次插入 'x' -> local_table['x']=5 -> 正常
第 10次插入 'x' -> local_table['x']=10 -> 正常
第 15次插入 'x' -> local_table['x']=15 -> 正常
第 20次插入 'x' -> local_table['x']=20 -> 正常
第 25次插入 'x' -> local_table['x']=25 -> 正常
第 30次插入 'x' -> local_table['x']=30 -> 正常

--- 阶段2后范围查询 'b' ~ 'p' ---
/home/liziheng/Desktop/lab4/client1.py:125: DeprecationWarning: 'passwd' is deprecated, use 'password'
Search('b', 'p')
ciphertext: iauoDqK8QI5mWlYUHYf17FOR0tvF7T1RURyTFdXxkklHq9aPngmjD6PPf3GYjUqT plaintext: banana
ciphertext: Aox5WioHVCvBNFG/4W4Lza3ANouYs8HBQJJ1z/8ha7JhJoJSMcf2jvbdYaPp1o plaintext: orange
ciphertext: fqPrkO9+Pgouk7ia9KBAeDq6FEqFdU0GRVOHl2vz9zwDoGNSjUQn07CyezurpJJb plaintext: cherry
ciphertext: YjA8mLkhDFCCFGS+offyORMUFM+OWIOh16c5If2C37dsEE8otrFcnrA2FvVmK0o5 plaintext: cherry
ciphertext: 4gsreZmD0YwE17XnFDHfDE0s7dw5DU4ZMcMU0vq6vW/9tYDL2RYgbiazPV5dFQ6 plaintext: orange
```

图 3: 阶段 2：重复插入‘x’及范围查询

5.3 阶段 3 & 4：编码分布观察与最终结果

阶段 3 查询数据库编码分布后，阶段 4 继续插入 20 次‘y’，最终编码分布如表1所示。从结果可以清楚地看到：

- 两个”apple”记录的编码值分别为 2.88×10^{17} 和 5.76×10^{17} ，相差一倍，攻击者无法从编码推断它们对应同一明文。
- 30 条”x”记录的编码分布在 8.83×10^{17} 到 1.13×10^{18} 之间，编码之间呈现明显的等间隔分布（每个间隔约 1.07×10^{16} ），频率信息被完全隐藏。
- 不同明文的编码保持了正确的字典序：apple < banana < cherry < orange < pear < x < y，与 CalPos 按字典序计算位置的逻辑完全一致，保序性得到了保证。
- 阶段 4 插入的 20 条’y’记录的编码位于最右侧（ $1.14 \times 10^{18} \sim 1.15 \times 10^{18}$ ），因为’y’在字典序中最大，被分配到末尾区域。

值得注意的是，整个测试过程中没有发生任何一次编码树分裂。总记录数仅 50 条（apple×2 + pear×1 + banana×1 + orange×2 + cherry×2 + x×30 + y×20），远小于节点容量阈值 128，编码区间 $[0, 2^{62}]$ 也绰绰有余。这在实验上是合理的——Recode 和分裂是极端负载下的边界行为，正常规模的数据插入不会触发。

表 1: 数据库编码分布概览

明文	出现次数	编码范围（近似）
apple	2	$2.88 \times 10^{17} \sim 5.76 \times 10^{17}$
banana	1	7.21×10^{17}
cherry	2	$7.39 \times 10^{17} \sim 7.57 \times 10^{17}$
orange	2	$7.75 \times 10^{17} \sim 7.93 \times 10^{17}$
pear	1	8.65×10^{17}
x	30	$8.83 \times 10^{17} \sim 1.13 \times 10^{18}$
y	20	$1.14 \times 10^{18} \sim 1.15 \times 10^{18}$

由于阶段 3 和阶段 4 的终端输出较长（共 110 余行），截图分为多张。图4用子图方式将阶段 3 的编码分布输出拼接展示，图5展示阶段 4 插入’y’后的最终数据库编码分布。

```

lizheng@lizheng-virtual-machine: ~/Desktop/lab4
阶段3: 大量重复插入后观察数据库编码分布
encoding=288239376511744 -> plaintext='apple'
encoding=576468752303423488 -> plaintext='apple'
encoding=72657940379279360 -> plaintext='banana'
encoding=7385983888961344 -> plaintext='cherry'
encoding=75604737398243328 -> plaintext='cherry'
encoding=774619135967725312 -> plaintext='orange'
encoding=792633534417207296 -> plaintext='orange'
encoding=84691128455235232 -> plaintext='pear'
encoding=882705526964617216 -> plaintext='x'
encoding=884957326778362464 -> plaintext='x'
encoding=88780926591587712 -> plaintext='x'
encoding=891712726219358208 -> plaintext='x'
encoding=896216325846728704 -> plaintext='x'
encoding=908719925474899200 -> plaintext='x'
encoding=909272134738848192 -> plaintext='x'
encoding=918734323983581184 -> plaintext='x'
encoding=936748722493863168 -> plaintext='x'
encoding=954763212602545152 -> plaintext='x'
encoding=963770320257286144 -> plaintext='x'
encoding=97277519512827136 -> plaintext='x'
encoding=108086316538991104 -> plaintext='x'

```

(a) 阶段 3 编码分布（一）

```

lizheng@lizheng-virtual-machine: ~/Desktop/lab4
encoding=97277519512827136 -> plaintext='x'
encoding=108086316538991104 -> plaintext='x'
encoding=1013389916158361600 -> plaintext='x'
encoding=1017813515785732896 -> plaintext='x'
encoding=102231715412103592 -> plaintext='x'
encoding=1026820715048473888 -> plaintext='x'
encoding=1035827914295214880 -> plaintext='x'
encoding=104083513549955072 -> plaintext='x'
encoding=105384231284696864 -> plaintext='x'
encoding=1058345912432066560 -> plaintext='x'
encoding=1060597712245751808 -> plaintext='x'
encoding=10617236125594432 -> plaintext='x'
encoding=1062649512059437056 -> plaintext='x'
encoding=1067353111686887552 -> plaintext='x'
encoding=1073550713154178848 -> plaintext='x'
encoding=1080863918568919040 -> plaintext='x'
encoding=1116892707587883808 -> plaintext='x'
encoding=1123899980842624000 -> plaintext='x'
encoding=1134987186097364992 -> plaintext='x'

```

(b) 阶段 3 编码分布（二）

图 4: 阶段 3: 大量重复插入后的数据库编码分布


```
encoding=1134907106097364992 -> plaintext='x'
阶段4：再持续插入'y' 触发更多分裂
=====
split = Insert('y')
插入'y' 第1次 -> local_table['y']<5 -> 正常
插入'y' 第16次 -> local_table['y']<10 -> 正常
插入'y' 第17次 -> local_table['y']<15 -> 正常
插入'y' 第20次 -> local_table['y']<20 -> 正常
-----
最终数据库编码分布
-----
encoding=28823037615171744 -> plaintext='apple'
encoding=576460751303423488 -> plaintext='apple'
encoding=720775040379239360 -> plaintext='banana'
encoding=73859033888761344 -> plaintext='cherry'
encoding=756604737398243328 -> plaintext='cherry'
encoding=774619135507723312 -> plaintext='orange'
encoding=792633344172072560 -> plaintext='orange'
encoding=864691128455135232 -> plaintext='pear'
encoding=88270526964617216 -> plaintext='x'
encoding=884957216778302464 -> plaintext='x'
```

(a) 最终编码分布 (一)

```
encoding=887209126591987712 -> plaintext='x'
encoding=891712726219358208 -> plaintext='x'
encoding=896216235846728704 -> plaintext='x'
encoding=900719925474099200 -> plaintext='x'
encoding=909772124728840192 -> plaintext='x'
encoding=918732323083581184 -> plaintext='x'
encoding=936748722493063168 -> plaintext='x'
encoding=954763121002454152 -> plaintext='x'
encoding=96377820257286144 -> plaintext='x'
encoding=97277519515027136 -> plaintext='x'
encoding=1008806316539991104 -> plaintext='x'
encoding=1013309916158361600 -> plaintext='x'
encoding=101781351785732896 -> plaintext='x'
encoding=1022317151413102592 -> plaintext='x'
encoding=1026620715040473088 -> plaintext='x'
encoding=103502794295214080 -> plaintext='x'
encoding=1044835113549955072 -> plaintext='x'
encoding=1053842312084696864 -> plaintext='x'
encoding=1058345912430655008 -> plaintext='x'
encoding=1068597122425731808 -> plaintext='x'
encoding=1061723612152594432 -> plaintext='x'
encoding=106284951059437056 -> plaintext='x'
encoding=106735111668007552 -> plaintext='x'
```

(b) 最终编码分布 (二)

```
encoding=1071856711314170048 -> plaintext='x'
encoding=1080803918568919040 -> plaintext='x'
encoding=11689270158783008 -> plaintext='x'
encoding=11258990842624000 -> plaintext='x'
encoding=1134907106097364992 -> plaintext='x'
encoding=113470850869706304 -> plaintext='y'
encoding=1135751531027496960 -> plaintext='y'
encoding=1136033006004207616 -> plaintext='y'
encoding=1136595953957628928 -> plaintext='y'
encoding=113715890591850240 -> plaintext='y'
encoding=1139410705724735488 -> plaintext='y'
encoding=1139973655678150800 -> plaintext='y'
encoding=114025513065867456 -> plaintext='y'
encoding=114039586814322784 -> plaintext='y'
encoding=1140536605631578112 -> plaintext='y'
encoding=1141662505538420736 -> plaintext='y'
encoding=114222455491842048 -> plaintext='y'
encoding=1142788405445263360 -> plaintext='y'
encoding=1143914305352105984 -> plaintext='y'
encoding=1144477253305272392 -> plaintext='y'
encoding=1145040205258948608 -> plaintext='y'
encoding=1146166105165791232 -> plaintext='y'
encoding=1146417964979476480 -> plaintext='y'
```

(c) 最终编码分布 (三)

```
encoding=1136595959597628928 -> plaintext='y'
encoding=113715890591850240 -> plaintext='y'
encoding=1139410705724735488 -> plaintext='y'
encoding=1139973655678150800 -> plaintext='y'
encoding=114025513065867456 -> plaintext='y'
encoding=114039586814322784 -> plaintext='y'
encoding=1140536605631578112 -> plaintext='y'
encoding=1141662505538420736 -> plaintext='y'
encoding=114222455491842048 -> plaintext='y'
encoding=1142788405445263360 -> plaintext='y'
encoding=1143914305352105984 -> plaintext='y'
encoding=1144477253305272392 -> plaintext='y'
encoding=1145040205258948608 -> plaintext='y'
encoding=1146166105165791232 -> plaintext='y'
encoding=1146417964979476480 -> plaintext='y'
encoding=115066978479161728 -> plaintext='y'
encoding=115172506470004352 -> plaintext='y'
测试完成！共发生 0 次编码树分裂
最终 local_table = {'apple': 2, 'pear': 1, 'banana': 1, 'orange': 2, 'cherry': 2, 'x': 30, 'y': 20}
```

(d) 最终编码分布 (四)

图 5: 阶段 4: 插入'y' 后的最终数据库编码分布

5.4 运行中的问题分析与修复

在首次运行原始代码时，阶段 3 末尾出现了异常：

```
1 ValueError: Padding is incorrect.
```

错误发生在客户端 AES 解密时，PKCS7 去填充校验失败，说明拿到的密文无法用当前密钥正确解密。

排查后定位到两个问题：

1. **重复调用 FHInsert**。client.py 的 Insert 函数中，原代码先执行了 `select FHInsert(...)` 来获取返回值，紧接着又调用 `call pro_insert(...)`，而存储过程内部又会调用一次 FHInsert。这导致同一条密文被插入了 B+ 树两次——B+ 树的记录数和数据库表的记录数不一致，随着插入增多，两者之间的状态偏差越来越严重，最终在查询编码时出现了无法解密的情况。修复方法是去掉第一次 FHInsert 调用，改为在存储过程执行后通过 `FHStart()` 的返回值判断是否发生了 Recode。
2. **数据库残留数据**。client.py 启动时没有清理 example 表中的旧数据。如果上一次运行使用了不同的 AES 密钥 (key 和 base_iv 是程序每次启动时随机生成的)，旧密文自然无法解密。修复方法是在测试代码开头加上 `TRUNCATE TABLE example`。

修复后重新运行，程序正常完成所有 4 个阶段的测试，不再出现崩溃。

6 心得体会

这次实验让我对 FH-OPE 的理解从课本上的定义变成了可以亲手跑通的代码，有几个感受比较深的地方。

首先是”频率隐藏”和”保序”这两个看似冲突的目标是怎么被分开解决的。之前看教材一直觉得这两件事搅在一起很难想清楚，但写成代码之后就就很直观了——客户端管加密（随机化，隐藏频率），服务

器管编码 (B+ 树, 维持顺序), 两边各司其职, 合在一起才构成完整的方案。这种” 职责分离” 的思路在安全系统设计里应该很常见。

其次是 Recode 操作的设计确实巧妙。编码空间不够的时候不去硬塞, 而是拉上兄弟节点在更大范围里重新” 排排坐”, 同时对客户端和数据库来说几乎透明 (只需批量更新一次编码值)。当然代价也是有的——Recode 一旦触发, 要更新的记录可能很多。如果在生产环境中用这个方案, 怎么在” 空间利用率” 和”Recode 频率” 之间找一个平衡点, 是个值得琢磨的工程问题。

第三, 写 UDF 这件事让我对” 跑在数据库进程里的代码” 有了更直接的敬畏。UDF 运行在 MySQL 的进程空间内, 一次空指针或数组越界就直接把整个 MySQL 搞挂。实验中调试的时候 MySQL 崩溃了好几次, 每次都要重启服务, 确实挺折磨人的, 但也确实让人记住了写 C++ 时要多小心边界条件。

另外, 客户端的 `local_table` 实际上承担了” 明文 $\rightarrow$ 位置” 的映射, 它存储了频率信息。这提醒我 FH-OPE 的安全性建立在” 客户端可信、服务器不可信” 的模型之上——`local_table` 绝不能泄露给服务器, 否则频率隐藏就前功尽弃了。

总的来说, 这次实验除了让我掌握了 FH-OPE 的具体实现, 也锻炼了 C++ 和 Python 混合编程、MySQL UDF 开发这类比较” 跨界” 的工程能力。如果时间允许, 后续还可以试试多线程并发插入的安全性、测量 Recode 的性能开销、或者尝试不同的编码分配策略 (比如非均匀间隔) 来优化空间利用率。